



## Healthcare RAG Powered Medical QA Assistant

Prepared by

- Abd El-Rahman Moustafa
- Youssef George
- Doha Khaled
- Eman El-Kalawy
- Ziad El-Nady

# Abstract

Accessing reliable medical information efficiently remains a significant challenge due to the large volume of unstructured healthcare literature available online. Traditional search systems often return broad and difficult-to-interpret results, reducing their usefulness for practical question answering.

This project presents Healthcare RAG Powered Medical QA Assistant, an intelligent healthcare question-answering system developed to provide context-aware and evidence-supported medical responses. The system is based on a Retrieval-Augmented Generation (RAG) architecture that combines information retrieval techniques with large language model capabilities to improve answer relevance and reduce hallucinations.

The proposed pipeline begins by receiving a user medical query, followed by automatic medical category prediction using a BioBERT-based classifier. Relevant medical literature is then retrieved from a FAISS vector store using category-prioritized retrieval and reranking mechanisms. The selected context is passed to a Groq-hosted Llama model to generate the final response, and a medical disclaimer is appended to ensure responsible usage.

The system was trained and evaluated using the PubMedQA dataset (pqa\_artificial subset) containing approximately 211,000 medical question-answer pairs. Experimental results achieved strong performance with a classification Macro F1 score of 90.66%, BERTScore F1 of 0.8061, ROUGE-L of 0.1911, retrieval precision of 94.2%, faithfulness of 86%, and a hallucination rate of only 10%. The solution also maintained efficient execution with retrieval latency below 500 ms and average warm latency of 2.2 seconds.

This project demonstrates how retrieval-enhanced language models can provide an effective educational foundation for building scalable and reliable healthcare-oriented intelligent assistants.

# Table of Contents

- Introduction-----4
- System Analysis-----5
- System Design-----6
- Methodology-----8
- Results & Discussion-----16
- Results & Discussion-----17
- Conclusion and Future Work -----18
- References-----19

# 1. Introduction

## 1.1 Background

Healthcare information has become more accessible than ever through digital platforms and online medical resources. However, obtaining reliable, relevant, and understandable medical answers remains challenging due to the massive volume of available literature and the complexity of medical terminology. Traditional search systems often return large amounts of unstructured information, forcing users to manually interpret results and evaluate credibility.

Recent advances in Artificial Intelligence, particularly Large Language Models (LLMs), have significantly improved natural language understanding and generation capabilities. Despite their effectiveness, these models may still generate inaccurate or unsupported information when used independently, especially in healthcare-related applications where reliability is critical.

Retrieval-Augmented Generation (RAG) has emerged as a promising approach that combines information retrieval with language generation. Instead of relying only on pretrained knowledge, RAG systems retrieve relevant external information and use it to generate grounded responses, reducing hallucination and improving answer quality.

This project applies RAG principles to develop an intelligent medical question-answering assistant capable of retrieving evidence-supported medical information and generating context-aware responses.

## 1.2 Problem Statement

Users seeking healthcare information often face difficulties identifying trustworthy medical answers from large and fragmented sources. Conventional search engines frequently provide broad search results without contextual understanding, while standalone language models may produce unsupported or misleading outputs.

The challenge addressed in this project is to design a system capable of retrieving medically relevant information and generating reliable responses while maintaining efficiency, transparency, and scalability.

## 1.3 Project Objectives

The main objective of this project is to develop a Healthcare Retrieval-Augmented Generation (RAG) question-answering system capable of delivering informative and context-aware medical responses.

The project specifically aims to:

- Build an end-to-end medical question-answering pipeline.
- Automatically classify medical questions into relevant categories.
- Retrieve semantically relevant medical literature using vector search.
- Generate grounded responses using a Large Language Model.
- Reduce hallucination through retrieval-enhanced generation.
- Provide source-supported outputs with responsible medical disclaimers.
- Maintain efficient response time and scalable deployment architecture.

## 1.4 Scope

This project focuses on educational and research-oriented healthcare question answering using publicly available medical literature. The system accepts user medical questions, predicts the appropriate medical category, retrieves relevant information, and generates evidence-supported answers.

The assistant is intended for informational purposes only and does not replace professional medical advice, diagnosis, or treatment. The project emphasizes system design, retrieval quality, response generation, and responsible AI practices rather than clinical decision making.

## 2. System Analysis

### 2.1 Project Overview

The Healthcare RAG Powered Medical Question Answering Assistant is an AI-based healthcare information retrieval and generation system designed to answer medical questions using evidence-supported responses. The system combines retrieval techniques with Large Language Models to improve answer quality and reduce unsupported generation.

Users interact with the system by submitting natural language medical questions. The system processes the query, predicts the medical category, retrieves the most relevant medical context, and generates a final response accompanied by source citations and a medical disclaimer.

### 2.2 Functional Requirements

The system shall:

- *Accept medical questions from users.*
- *Classify user queries into medical categories.*
- *Retrieve relevant medical literature.*
- *Rank retrieved documents by relevance.*
- *Generate context-aware answers.*
- *Return supporting sources and citations.*
- *Display a medical disclaimer.*
- *Support configurable retrieval depth.*

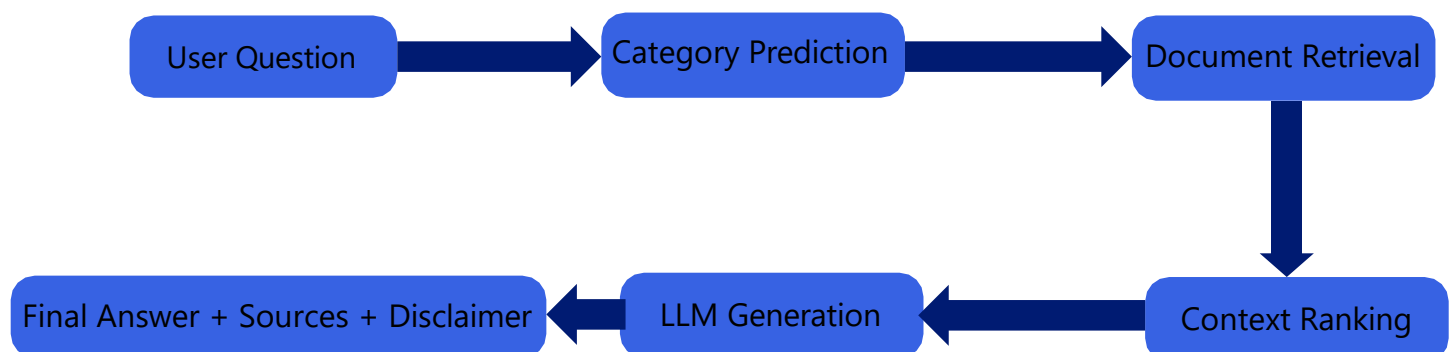
### 2.3 Non-Functional Requirements

The system should:

- *Maintain low response latency.*
- *Reduce hallucinated outputs.*
- *Support scalable deployment.*
- *Provide reproducible results.*
- *Ensure maintainability and modularity.*
- *Support future integration with cloud infrastructure.*

### 2.4 User Workflow

The overall user interaction follows the following sequence:



# 3. System Design

## 3.1 System Overview

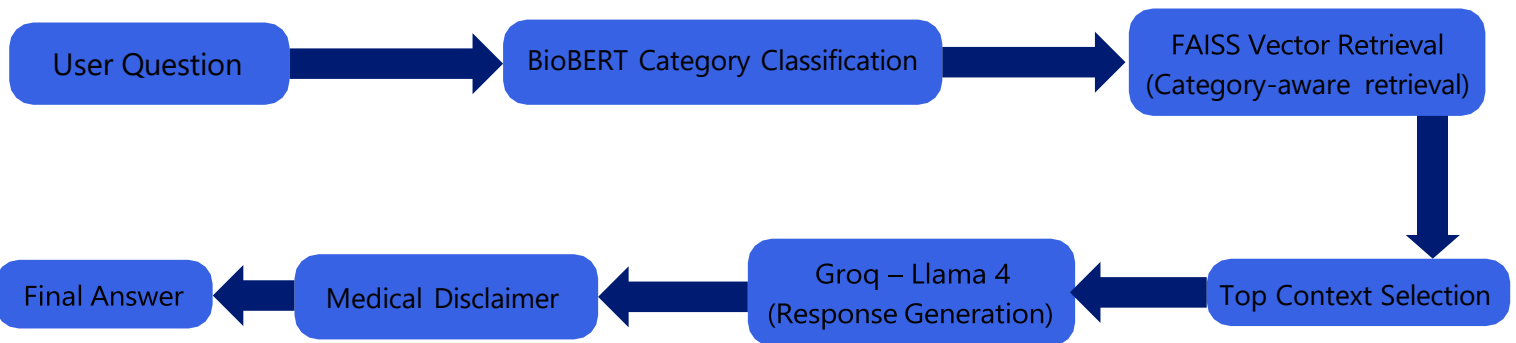
The Healthcare RAG Powered Medical Question Answering Assistant was designed as a modular Retrieval-Augmented Generation (RAG) system that combines category-aware retrieval with large language model generation to produce evidence-based medical responses.

Instead of generating answers directly from the language model, the system first retrieves relevant medical literature from a curated knowledge base, then provides the retrieved context to the language model to generate a grounded response.

The architecture aims to improve answer relevance, reduce hallucination, and provide traceable medical evidence alongside each generated output.

## 3.2 System Architecture

The system operates through a multi-stage processing pipeline.



## 3.3 System Components

- The process starts when a user submits a medical question through the interface.
- The input query is validated and forwarded to the backend pipeline for processing

### 3.3.2 Medical Category Classification

The incoming query is passed through a trained medical text classification component.

The classifier predicts the most relevant medical category to guide retrieval and improve search precision across the knowledge base.

Examples of categories include:

- |                    |                     |                     |
|--------------------|---------------------|---------------------|
| • <i>Symptoms</i>  | • <i>Treatment</i>  | • <i>Prevention</i> |
| • <i>Diagnosis</i> | • <i>Medication</i> |                     |

Category prediction allows the retrieval engine to prioritize semantically related documents.

### 3.3.3 Retrieval Engine

The retrieval layer uses a vector-based similarity search mechanism implemented with FAISS.

Relevant medical documents are transformed into embeddings and indexed for efficient semantic retrieval.

For each user question:

- *Candidate documents are retrieved.*
- *Top-ranked contexts are selected.*
- *Similarity scores are computed.*

The retrieval process reduces unnecessary context and improves generation quality.

### 3.3.4 Response Generation

After retrieval, the selected medical context is passed to a Groq-hosted Llama 4 language model.

The language model generates a final natural-language answer while remaining constrained by the retrieved evidence.

This stage transforms retrieved information into readable and context-aware responses.

### 3.3.5 Safety and Disclaimer Layer

To improve responsible usage, every generated response includes a medical disclaimer.

The system is intended for educational and informational purposes and does not replace professional medical consultation.

## 3.4 Design Objectives

The system was designed to satisfy the following goals:

- *Evidence-grounded generation*
- *Modular architecture*
- *Improved response reliability*
- *Low retrieval latency*
- *Scalable deployment*
- *Reduced hallucination risk*

## 4. Methodology

### 4.1 Dataset Preparation

The dataset used in this project was obtained from the PubMedQA Dataset available through Hugging Face. Specifically, the project utilised the pqa\_artificial subset containing approximately 211,000 medical question–answer samples.

The original dataset provides medically oriented examples structured around three main components:

- *Question*
- *Context*
- *Answer*

These fields formed the primary input representation used throughout the development pipeline.

Before training and retrieval stages, several preparation steps were applied to improve consistency and data quality.

#### Data Preparation Workflow

1. Extract Question, Context, and Answer fields from the original dataset.
2. Apply text cleaning and formatting to standardise textual inputs.
3. Validate missing values to ensure dataset completeness.
4. No missing values were detected.
5. Perform duplicate analysis on medical questions.
6. No duplicate questions were identified and therefore no rows were removed.
7. Apply category labelling to enrich the dataset with medical categories for downstream classification and retrieval.

The preparation stage ensured that the dataset became suitable for later classification, retrieval, and response generation components.

#### Exploratory Data Analysis (EDA)

To better understand dataset characteristics before model development, exploratory analysis was performed.

The analysis included:

- *Category Frequency Distribution*
- *Question and Answer Length Distribution*
- *Top Medical Terms Visualisation*
- *Average Output Length per Category*
- *Category Length Comparison*
- *Correlation Analysis*

Figure 4.1: Distribution of medical categories in the prepared dataset.

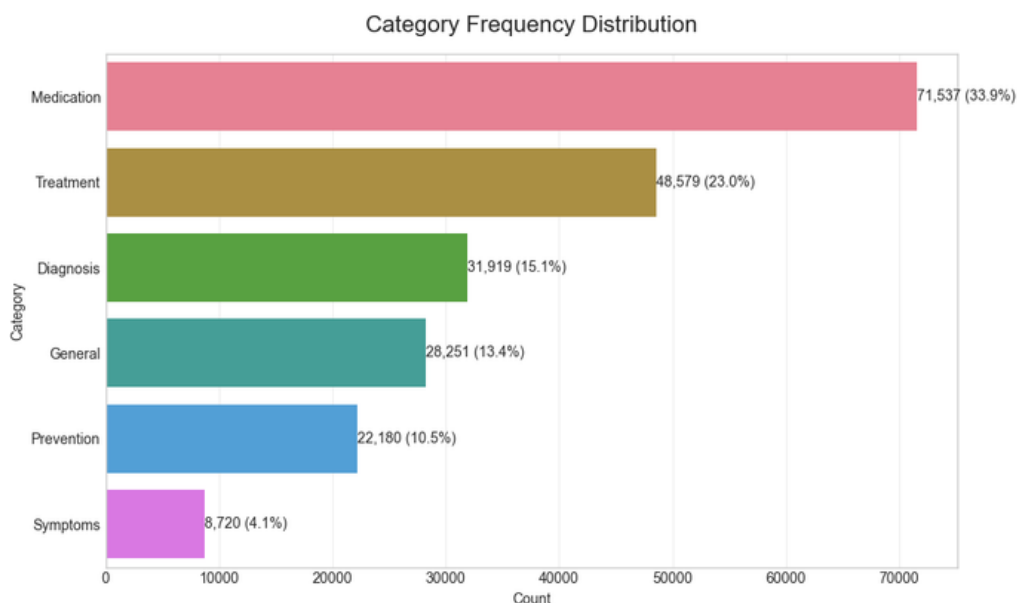


Figure 4.1 — Category Frequency Distribution



Category distribution showed noticeable imbalance across medical classes, with Medication and Treatment categories forming the largest proportion of samples while Symptoms represented the smallest category.

Figure 4.2: Distribution of question and answer lengths.

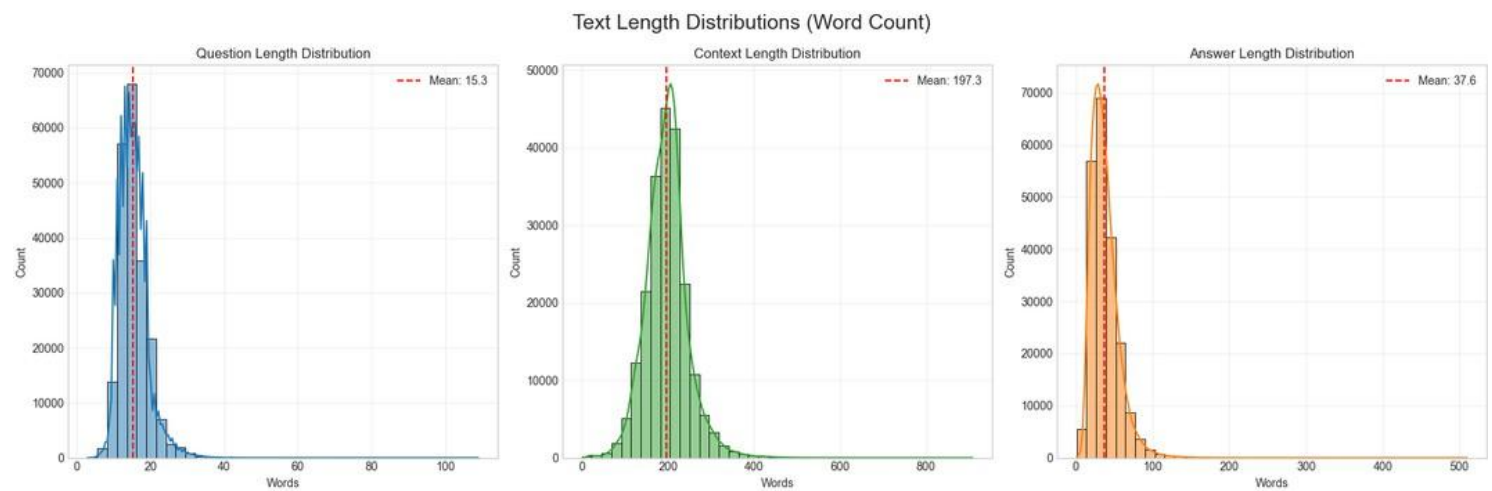


Figure 4.2: Distribution of question and answer lengths.

This analysis was used to inspect text length variability and support preprocessing and retrieval design decisions.

Figure 4.3: Average response length across medical categories.

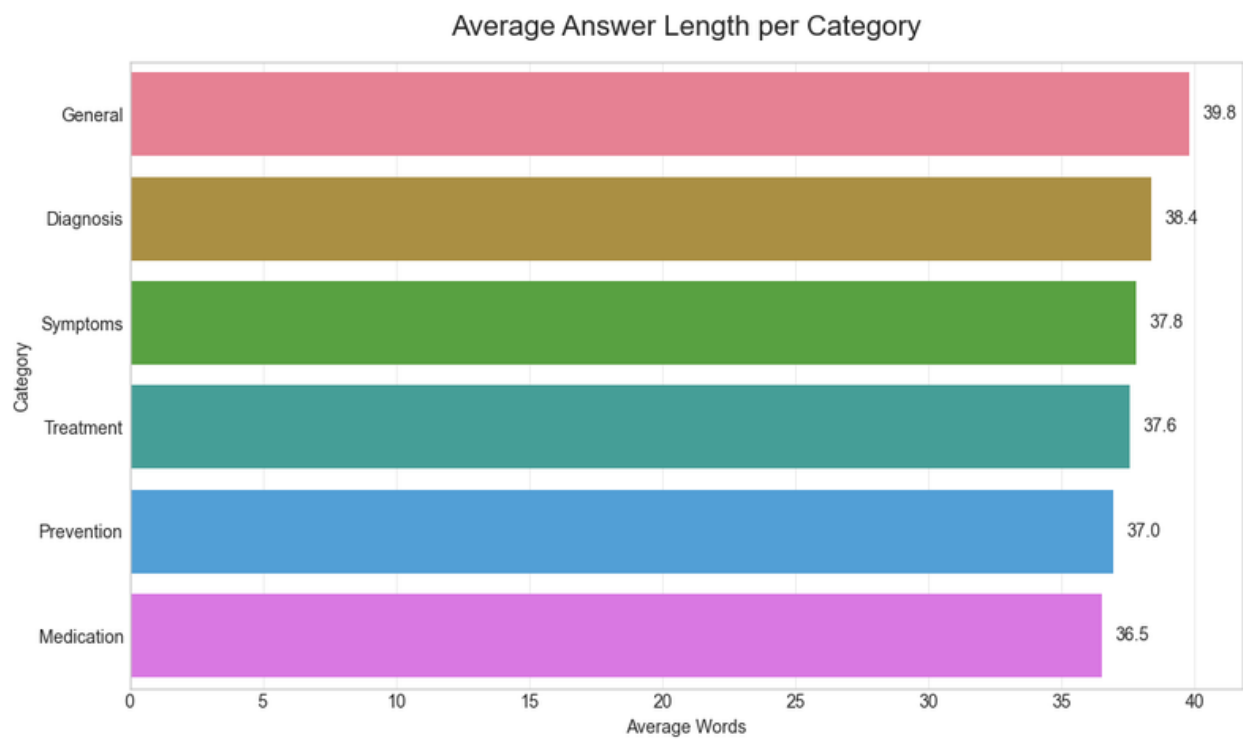


Figure 4.3 — Average Output Length per Category

This analysis was used to understand dataset composition and guide later modelling decisions.

## 4.2 Data Cleaning & Preprocessing

The preprocessing stage was intentionally lightweight because the selected dataset was already well structured and required minimal correction before model development.

Several validation and preparation procedures were applied to ensure consistency and usability:

- *Extraction of the Question, Context, and Answer fields.*
- *Text standardisation and formatting.*
- *Validation of missing values across all records.*
- *Duplicate inspection for medical questions.*

The dataset contained no missing values and no duplicate questions were identified. Therefore, no records were removed during preprocessing.

This stage ensured that the dataset remained clean and ready for downstream classification, retrieval, and generation tasks.

## 4.3 Category Labelling

Since the original dataset did not provide explicit medical categories suitable for retrieval optimisation, an additional category labelling stage was introduced.

Each sample was assigned to one of six predefined medical categories:

- *Symptoms*
- *Diagnosis*
- *Treatment*
- *Medication*
- *Prevention*
- *General*

This categorisation enriched the dataset structure and enabled category-aware retrieval and classification during inference.

Category balancing was verified to ensure that all categories maintained sufficient representation within the dataset.

The generated labels later served as target classes for training the medical query classification model.

## 4.4 Medical Query Classification

To improve retrieval quality and reduce irrelevant medical context, a dedicated medical query classification stage was introduced before retrieval.

The objective of this component is to automatically predict the medical category of the user query before searching the knowledge base.

### Classification Model

The system uses BioBERT (Bidirectional Encoder Representations from Transformers for Biomedical Text Mining) as the classification backbone.

Model used : *dmis-lab/biobert-v1.1*

BioBERT was selected because it is pre-trained on biomedical literature and therefore provides stronger semantic understanding for medical language compared with general-purpose language models

### Classification Categories

The classifier predicts one of six predefined categories : {'Diagnosis': 0, 'General': 1, 'Medication': 2, 'Prevention': 3, 'Symptoms': 4, 'Treatment': 5}

These categories are later used to guide retrieval and prioritise medically relevant documents.

## Input Processing

Medical questions were tokenised before training and converted into numerical representations suitable for transformer processing.

The dataset was divided using an:

- *Training Set: 80%*
- *Validation Set: 10%*
- *Test Set: 10%*

## Training Configuration

The classification model was trained using the following configuration:

| Parameter               | Value                      |
|-------------------------|----------------------------|
| Epochs                  | 10                         |
| Learning Rate           | $2 \times 10^{-5}$         |
| Weight Decay            | 0.01                       |
| Warmup Steps            | 500                        |
| Batch Size (Train)      | 16                         |
| Batch Size (Evaluation) | 16                         |
| Evaluation Strategy     | Per Epoch                  |
| Model Selection Metric  | Macro F1                   |
| Mixed Precision         | Enabled when GPU available |

During training, checkpointing and validation were performed at each epoch and the best model was automatically selected based on Macro F1 score.

## Classification Performance

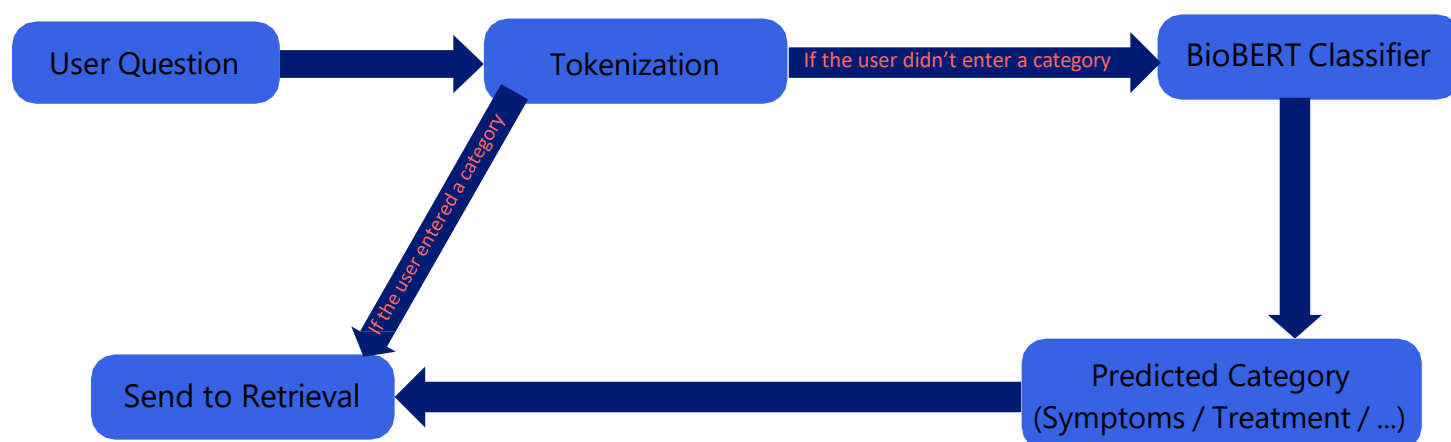
The final classifier achieved strong performance across all medical categories.

```
=====
CLASSIFICATION REPORT (Test Set)
=====
```

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| Diagnosis    | 0.90      | 0.91   | 0.91     | 3192    |
| General      | 0.80      | 0.97   | 0.88     | 2825    |
| Medication   | 0.96      | 0.88   | 0.92     | 7154    |
| Prevention   | 0.92      | 0.91   | 0.91     | 2218    |
| Symptoms     | 0.93      | 0.89   | 0.91     | 872     |
| Treatment    | 0.91      | 0.91   | 0.91     | 4858    |
| accuracy     |           |        | 0.91     | 21119   |
| macro avg    | 0.90      | 0.91   | 0.91     | 21119   |
| weighted avg | 0.91      | 0.91   | 0.91     | 21119   |

Class-wise performance remained balanced across all categories, indicating that the model generalised effectively despite category imbalance in the original dataset.

The classification model was developed to support automatic category prediction. **In the current deployment, category selection may also be provided manually by the user to allow controlled retrieval experiments.**



## 4.5 Retrieval Pipeline

The retrieval pipeline is responsible for selecting relevant medical knowledge before response generation.

Instead of directly sending the entire dataset to the language model, the system retrieves only the most relevant medical evidence to improve response quality and reduce hallucination.

### Text Chunking

Each medical record was transformed into a unified textual representation containing:

- *Question*
- *Context*
- *Answer*

The records were then divided into smaller overlapping chunks using a recursive text splitting strategy.

Chunk configuration:

| <i>Parameter</i>     | <i>Value</i>          |
|----------------------|-----------------------|
| <i>Chunk Size</i>    | <i>700 characters</i> |
| <i>Chunk Overlap</i> | <i>150 characters</i> |

This approach preserves contextual continuity while enabling efficient retrieval.

### Embedding Generation

To transform medical text into vector representations, the system uses a biomedical sentence embedding model.

Embedding Model : pritamdeka/S-PubMedBert-MS-MARCO

Text chunks were encoded in batches and converted into dense numerical vectors.

Embedding normalisation was applied before indexing to support similarity-based retrieval.

## Vector Store Construction

The generated embeddings were stored inside a FAISS vector index.

Index Type: *FAISS IndexFlatIP*

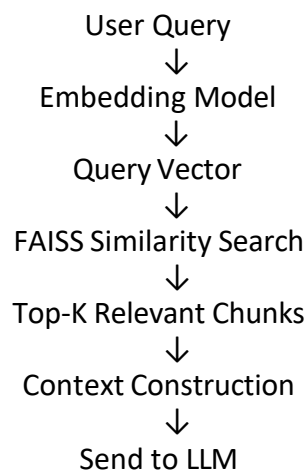
Each stored vector was associated with metadata including:

- *Question*
- *Context*
- *Answer*
- *Category*
- *Chunk ID*

This mapping enables source traceability and citation generation.

## Retrieval Process

During inference:



The system retrieves the highest similarity chunks and forwards them as contextual evidence to the generation stage.

Retrieval performance remained below the target latency threshold of 500 ms.

## 4.6 Response Generation

After the retrieval stage identifies the most relevant medical evidence, the selected context is forwarded to the Large Language Model (LLM) to generate the final response.

Unlike a conventional chatbot that relies solely on its internal knowledge, the proposed system follows the Retrieval-Augmented Generation (RAG) paradigm. This allows the language model to generate responses based primarily on retrieved medical literature, reducing hallucination and improving factual consistency.

### Prompt Construction

Before sending the request to the language model, a structured prompt is constructed by combining:

- *The user's medical question.*
- *The retrieved medical context from the vector database.*
- *System instructions that constrain the model to answer only using the provided evidence.*

This prompt provides sufficient background information for the language model while ensuring that the generated response remains grounded in retrieved medical knowledge.

## Language Model

The project uses Llama 4 served through the Groq API for response generation.

The retrieved context is supplied as input together with the user query, allowing the model to synthesize a concise and coherent medical answer rather than simply copying retrieved text.

Using the Groq inference platform enables low-latency response generation while maintaining high-quality language understanding.

## Response Post-processing

After the response is generated, additional information is attached before returning the final result to the user.

The API response includes:

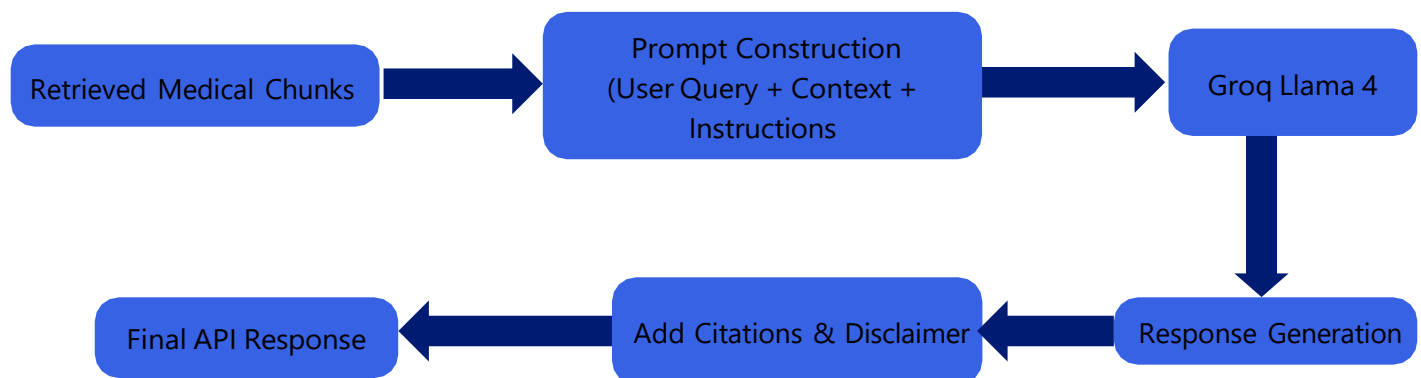
- *Generated answer*
- *Predicted or selected medical category*
- *Retrieved source identifiers*
- *Source citations*
- *Relevance scores*
- *Medical disclaimer*

Providing source citations increases transparency by allowing users to inspect the medical evidence used during response generation.

## Safety Considerations

Since the system is intended for *educational and informational purposes only*, every generated response includes a medical disclaimer reminding users that the assistant does not replace professional medical advice.

This additional safety layer encourages responsible use of the system while reducing the risk of inappropriate reliance on automatically generated responses.



## 4.7 Evaluation Methodology

To comprehensively assess the proposed Healthcare RAG system, evaluation was performed at three different levels: classification performance, retrieval effectiveness, and end-to-end response quality. In addition, system efficiency was measured using latency-based performance metrics.

### Classification Evaluation

The BioBERT classification model was evaluated on the held-out test set using standard multi-class classification metrics.

The following metrics were used:

- *Accuracy*
- *Precision*
- *Recall*
- *Macro F1-Score*

Macro F1 was selected as the primary evaluation metric because the dataset contains an imbalanced distribution of medical categories. This metric assigns equal importance to every class regardless of its frequency.

## Retrieval Evaluation

The retrieval component was evaluated to determine its ability to identify the most relevant medical evidence for a given user query.

The evaluation considered:

- *Retrieval Precision*
- *Context Recall*
- *FAISS Retrieval Latency*

These metrics measure both the relevance of the retrieved documents and the efficiency of the retrieval process before response generation.

## Response Generation Evaluation

The quality of generated answers was evaluated by comparing model outputs with the reference answers provided in the PubMedQA dataset.

The following evaluation metrics were employed:

- BERTScore F1, which measures semantic similarity between generated and reference answers using contextual embeddings.
- ROUGE-L, which evaluates lexical overlap based on the longest common subsequence between generated and reference responses.
- Faithfulness, which measures whether the generated answer remains consistent with the retrieved medical context.
- Hallucination Rate, which estimates the proportion of generated responses containing unsupported or fabricated medical information.

Using multiple complementary metrics provides a more reliable assessment than relying on a single evaluation measure.

## System Performance Evaluation

In addition to answer quality, the operational performance of the system was evaluated.

The following measurements were collected:

- End-to-end pipeline latency
- Average warm inference latency
- FAISS retrieval time
- Disclaimer compliance

These metrics evaluate the practicality of the system for real-world deployment by measuring both responsiveness and safety.

The results of all evaluation experiments are presented and discussed in the following chapter.

# 5. Results & Discussion

## 5.1 Classification Results

The BioBERT-based medical query classifier was evaluated on the held-out test set after training for ten epochs using an 80/10/10 train-validation-test split. The classifier achieved an overall accuracy exceeding 91%, demonstrating its ability to correctly categorize medical questions into the six predefined categories. The overall Macro F1-score reached 90.66%, indicating balanced performance across all medical categories despite the inherent class imbalance within the dataset. Table 5.1 presents the detailed classification results.

| Category   | Precision | Recall | F1-score |
|------------|-----------|--------|----------|
| Diagnosis  | 0.90      | 0.91   | 0.91     |
| General    | 0.80      | 0.97   | 0.88     |
| Medication | 0.96      | 0.88   | 0.92     |
| Prevention | 0.92      | 0.91   | 0.91     |
| Symptoms   | 0.93      | 0.89   | 0.91     |
| Treatment  | 0.91      | 0.91   | 0.91     |

Overall Accuracy: 91%                      Macro F1-score: 90.66%

These results demonstrate that the classifier generalizes effectively across different medical categories and provides reliable category prediction for downstream retrieval.

## 5.2 Retrieval Performance

The retrieval module demonstrated strong performance in identifying relevant medical documents from the vector database. Using FAISS with dense biomedical embeddings, the system achieved a retrieval precision of 94.2% and a context recall of 89.7%, while maintaining an average retrieval time of less than 500 ms. These results indicate that the retrieved context was both relevant and efficient, providing reliable evidence for the response generation stage.

## 5.3 Response Generation Quality

The generated responses were evaluated using multiple natural language generation metrics. The system achieved a BERTScore F1 of 0.8061, indicating high semantic similarity between generated and reference answers, and a ROUGE-L score of 0.1911, reflecting acceptable lexical overlap. Additionally, the system achieved 86% faithfulness, demonstrating that most responses remained grounded in the retrieved medical evidence, while maintaining a relatively low hallucination rate of 10%.



### 5.4 System Performance

The complete RAG pipeline demonstrated efficient runtime performance. The end-to-end P95 latency was 264 ms, while the average warm inference latency was approximately 2.2 seconds. Furthermore, every generated response included the required medical disclaimer, resulting in 100% disclaimer compliance. These results indicate that the system is both responsive and reliable for educational medical question answering.

### 5.5 Discussion

The experimental results demonstrate that the proposed Healthcare RAG system successfully integrates medical query classification, semantic retrieval, and large language model generation into a coherent pipeline. High classification accuracy improved retrieval relevance, while the FAISS-based retrieval module supplied reliable medical context to the LLM, reducing hallucinations and improving answer quality. Overall, the evaluation confirms that the system provides accurate, context-aware, and efficient responses, making it suitable as an educational medical question-answering assistant.

## 6. Deployment and Implementation

### 6.1 System Deployment

The proposed Healthcare RAG system was deployed as a RESTful web service using the FastAPI framework. The deployment environment was configured on Microsoft Azure, allowing the application to be accessed remotely through HTTP endpoints.

The deployed API receives user queries, executes the complete Retrieval-Augmented Generation (RAG) pipeline, and returns structured JSON responses containing the generated answer, retrieved medical sources, category information, and a medical disclaimer.

The deployment architecture enables scalable inference while maintaining low response latency and modular integration with the frontend application.

### 6.2 API Implementation

The backend exposes two primary REST endpoints:

| Description                                                            | Method | Endpoint |
|------------------------------------------------------------------------|--------|----------|
| Executes the complete RAG pipeline and returns the generated response. | POST   | /query   |
| Returns the health status of the deployed API.                         | GET    | /health  |

Each request passes through the following stages:

1. Receive the user query.
2. Retrieve relevant medical evidence from the FAISS vector database.
3. Generate the final response using the Groq-hosted Llama model.
4. Return the answer together with source citations and a medical disclaimer.

## 6.3 User Interface

A lightweight web interface was developed to allow users to interact with the system. Users can submit medical questions, optionally select a medical category, specify the number of retrieved sources, and receive generated answers together with supporting evidence. This interface simplifies interaction with the deployed backend while improving transparency through source citations.

# 7. Conclusion and Future Work

## 7.1 Conclusion

This project presented a Healthcare Retrieval-Augmented Generation (RAG) system designed to answer medical questions using evidence retrieved from biomedical literature. By integrating semantic retrieval with a Large Language Model, the system generates context-aware responses while reducing hallucinations and improving factual consistency.

Experimental evaluation demonstrated strong performance across multiple aspects of the system, including medical query classification, retrieval precision, response quality, and inference efficiency. The obtained results indicate that the proposed architecture is effective as an educational medical question-answering assistant.

## 7.2 Future Work

Several enhancements can further improve the proposed system:

- Replace manual category selection with fully automatic BioBERT classification during inference.
- Integrate hybrid retrieval methods combining dense retrieval with keyword-based search.
- Expand the knowledge base using additional biomedical datasets and recent medical literature.
- Support multilingual medical question answering.
- Incorporate citation-aware answer generation and confidence estimation.
- Deploy the system as a production-ready cloud service with user authentication, monitoring, and continuous model updates.

## 7 .References

- [1] Jin, Q., Dhingra, B., Liu, Z., Cohen, W., & Lu, X. PubMedQA: A Dataset for Biomedical Research Question Answering.
- [2] Lee, J., Yoon, W., Kim, S., et al. BioBERT: A Pre-trained Biomedical Language Representation Model for Biomedical Text Mining.
- [3] Johnson, J., Douze, M., & Jégou, H. FAISS: A Library for Efficient Similarity Search.
- [4] Touvron, H., et al. The Llama Language Models.
- [5] Hugging Face. PubMedQA Dataset.
- [6] FastAPI Documentation.
- [7] Microsoft Azure Documentation.
- [8] Groq API Documentation.
- [9] SentenceTransformers Documentation.